

Conception d'un bot Discord

https://gemini.google.com/app/222ac908ba884dd

User prompt: # Projet Bot Discord ## Besoin Initial Il faut développer un bot Discord pour notre association de jeu (wargame, société, JdR, TCG) pour préparer les soirée jeu. Le bot va avoir pour mission de mettre en avant les joueurs inscrits et disponible pour un jeu à une date précise. Une fois les joueurs indiqué comme disponible pour jouer à un ou plusieurs jeux. Il faut qu'il puisse valider une "table" en indiquant quels joueurs vont jouer au même jeu sur la même table. (On peu avoir 2 table de SdA en 1v1 ou une table de SdA en 2v2, ou même une table de SdA en 1v1v1v1). Le Bot va devoir gérer des commande pour deux type de rôle. Admin et User. Nous reviendrons sur les différences plus tard. ## Stock Technique Pour faciliter la gestion et la maintenance, le projet sera fait en C#.NET10. Il sera hébergé sur une raspberry pi en local dans un premier temps. Il faudra l'organiser en clean architecture pour faciliter sa maintenance. Discord.NET est un wrapper util pour le développement. ## Fonctionnalité Le bot à pour mission de proposer sur un channel défini, à interval défini un formulaire de réservation de table qui va s'articuler en 2 temps. Dans un premier temps les joueurs pourront se déclarer disponible pour jouer à un ou plusieurs jeu via une commande. Dans un second temps les joueurs pourront faire des association de joueurs pour créer des table. Par exemple A se dit disponible pour jouer à 1 ou 2, B se dit disponible pour jouer à 2 ou 3. A et B se mettent d'accord pour jouer ensemble à 2 et créé donc une table ou A et B jouerons à 2. On peu aussi envisager que le joueur C, rejoigne la table de A et B en tant que joueurs ou en tant que spectateur. Le Bot doit afficher une Card qui s'actualise en fonction des actions des Users pour indiquer les joueurs disponible, absent, assigné à une table etc. Il faut qu'il soit possible de cancel une table pour se mettre absent ou se réassigné ailleurs. Un jeu "Autre" avec un champ de saisie libre doit toujours être disponible dans la liste des jeu, pour ne pas être vérouillé sur les reservation. Pour faciliter l'ergonomie il faudrait peu être envisager de créer une table en renseignant un jeu et des participant sans que ceux si soit forcément déjà inscrit en tant que disponible. L'appli est avant tout là pour avoir une vu exhaustive des réservations de la soirée et la partie échange/organisation peu se faire en dehors. ## Persona Admin Les utilisateurs possédant un certain rôle (à définir) seront considéré comme Admin. Un Admin possède des droits et commande étendu sur le Bot mais aussi tout les droits d'utilisation d'un User. - L'Admin à la possibilité de choisir dans quel channel le bot intervient. - L'Admin à la possibilité de définir à quel fréquence et à quel moment le bot propose un nouveau formulaire de réservation. - L'Admin à la possibilité de définir, maintenir et actualiser une liste de jeu disponible. ## Persona User - Un User peu se déclarer disponible pour un/plusieurs jeu présent dans la liste de jeu disponible. - Un User peu se déclarer à une table d'un jeu en tant que joueur ou Observateur. - Un User ne peu être déclarer qu'à une et une seul table. - Une table peu accueillir 2 à n joueurs. - Un joueur peu annuler une table pour s'assigner absent ou à une autre table. Je te demande :- Identifier les besoin et fonctionnement du projet - Questionner les point qui semble obscure, incomplet ou si l'intention n'est pas clairement défini. - Proposer des recommandations Technique pertinente pour le projet, que ce soit sur l'aspect fonctionnel, applicatif ou IA (MCP, Skills, etc) - Préparer un document technique permettant le développement du projet, que ce soit par un humain ou une IA. Contient les recommandations. Ainsi que la marche à suivre pour le déploiement et la mise en fonctionnement sur un Raspberri Pi Le document doit être suffisant complet et précis pour gérer ce projet de A à Z

Response: Analyse et décomposition atomique du besoin

Cadre et intention

Le projet formalise l'organisation asynchrone des sessions de jeu d'une association. L'intention centrale est d'éliminer les discussions dispersées en fournissant un point d'ancrage visuel unique (une « Card » ou message récapitulatif dynamique) dans Discord, permettant de passer de la déclaration d'intérêt individuel à la constitution effective de tables de jeu.

Décomposition des contraintes métier

- **Temporalité & Automatisation** : Génération récurrente et programmée d'une session de réservation sur un salon ciblé.
- **Catalogue de jeux** : Liste administrable de jeux prédéfinis, complétée obligatoirement par une option d'échappement « Autre » à saisie libre.
- **Gestion des disponibilités (Phase 1)** :
 - Association 1 . . . N entre un joueur et des jeux souhaités.
 - Possibilité d'indiquer un statut absent ou disponible.
- **Constitution des tables (Phase 2)** :
 - Une table regroupe 2 . . . N participants autour d'un jeu unique.
 - Distinction stricte des rôles au sein de la table : joueur ou Observateur.
 - Règle d'exclusivité : un utilisateur ne peut appartenir qu'à une seule table active pour une session donnée.
 - Capacité de composition directe : ajout de participants sans obligation de déclaration préalable de disponibilité.
- **Cycle de vie et modification** : Annulation d'une table, libération des places, réassignation, passage en statut absent.
- **Affichage dynamique** : Rendu visuel persistant (Embed Discord) reflétant l'état de la base de données en temps quasi-réel.

Audit critique et zones d'ombre

Avant d'entamer l'implémentation, huit points de friction logique et technique doivent être tranchés :

#	Point d'ombre / Risque	Impact	Solution préconisée
1	Cycle de vie de la session	Comment une session se clôture-t-elle ? Y a-t-il plusieurs sessions en parallèle (ex. vendredi soir et samedi après-midi) ?	Modéliser une entité GameSession avec date/heure explicite et statut (Draft, Open, Locked, Archived). Clôture automatique à l'heure H de l'événement.
2	Rate limits Discord (UI Updates)	Modifier un message Discord à chaque clic d'utilisateur déclenche l'erreur HTTP 429 (Too Many Requests).	Découpler l'écriture en base de l'affichage via un système de débounding (mise à jour du message différée de une à deux secondes regroupant les modifications).
3	Limite des 25 choix Discord	Les menus déroulants Discord (SelectMenu) sont limités à 25 entrées maximum.	Si le catalogue dépasse 24 jeux, utiliser un système de recherche via commande Slash avec autocomplétion ([Autocomplete]) ou pagination par modale.
4	Gouvernance d'une table	Qui a le droit de dissoudre une table ou d'en retirer un membre ? Le créateur ? N'importe quel membre ?	Seul le créateur de la table ou un Admin peut dissoudre la table. Un joueur peut uniquement se retirer lui-même. Si le retrait fait passer la table sous deux joueurs, la table passe en statut « Incomplète » sans être détruite immédiatement.
5	Consentement d'assignation directe	Si le joueur A crée une table et y inscrit de force le joueur B sans son accord préalable.	Autoriser l'assignation directe, mais notifier le joueur B par message éphémère ou mention avec un bouton d'annulation immédiate en un clic.
6	Statut d'Observateur	L'observateur compte-t-il dans l'exclusivité d'une seule table ?	Oui. Pour éviter les conflits d'horaires et d'attention, un utilisateur est soit joueur, soit observateur, mais rattaché à une unique table physique.
7	Stockage local Raspberry Pi	Une carte micro-SD supporte mal les écritures intensives non tamponnées.	Utiliser SQLite en mode WAL (Write-Ahead Logging) ou LiteDB avec synchronisation optimisée, avec backup automatisé sur un volume externe ou distant.
8	Persistance de l'interaction	Si le bot redémarre, comment gère-t-il les clics sur les anciens boutons de la Card ?	Utiliser des custom_id d'interaction sans état (ex. table:join:{sessionId}:{tableId}) analysés par un routeur global, plutôt que des écouteurs en mémoire éphémère.

Recommandations techniques

Architecture applicative

- **Clean Architecture** standardisée en quatre couches : Domain (règles pures, aucune dépendance externe), Application (cas d'usage, interfaces de commandes/requêtes via médiateur), Infrastructure (Entity Framework Core SQLite, Discord.Net, scheduler), et Bot (interactions, modales, embebs, configuration).
- **Persistance** : SQLite via **EF Core 10**. Léger, aucun démon lourd à faire tourner sur Raspberry Pi, portable via une unique fichier .db.
- **Planification** : Utilisation de **Quartz.NET** ou d'un IHostedService avec cron expression configurable pour la publication récurrente des messages d'organisation.
- **Mise à jour UI asynchrone** : Implémentation d'un System.Threading.Channels. Channel<Gui> consommé en tâche de fond pour réactualiser l'Embed de la session active avec un throttle de deux secondes.

Volet IA et intégrations avancées (MCP / LLM)

- **MCP Server (Model Context Protocol)** : Exposer un endpoint local (via studio ou SSE HTTP sécurisé) branché sur la couche Application. Cela permet à un agent IA connecté (ex. Claude Desktop ou assistant local) d'interroger la base du club : « Combien de joueurs sont dispo pour Warhammer 40k ce vendredi ? », « Crée une table pour le jeu Dune avec Alice et Bob ».
- **Traitement du jeu « Autre »** : Optionnel : intégrer un appel LLM très léger (via API externe ou petit modèle hébergé) pour catégoriser automatiquement les jeux saisis dans le champ libre « Autre » (ex. détecter que « 40k », « W40K » et « Warhammer » pointent vers le même jeu).

Spécification technique d'implémentation

Structure de la solution

```
src/
├── AssociationBot.Domain/
│   ├── Entities/
│   │   ├── Game.cs
│   │   ├── GameSession.cs
│   │   ├── Table.cs
│   │   ├── PlayerAvailability.cs
│   │   └── TableParticipant.cs
│   ├── Enums/
│   │   ├── ParticipantRole.cs # Player, Spectator
│   │   └── SessionStatus.cs # Draft, Open, Locked, Closed
│   └── Interfaces/
│       └── IUnitOfWork.cs
├── AssociationBot.Application/
│   ├── Common/
│   ├── Games/ # Commandes CRUD jeux
│   ├── Sessions/ # Création, refresh UI
│   └── Tables/ # Création, join, leave, cancel
├── AssociationBot.Infrastructure/
│   ├── Persistence/
│   │   ├── ApplicationDbContext.cs
│   │   └── Configurations/
│   ├── Scheduling/
│   │   └── SessionSchedulerService.cs
│   ├── Discord/
│   │   ├── DiscordUiThrottler.cs
│   │   └── DiscordMessageRenderer.cs
└── AssociationBot.Presentation/
    ├── Modules/
    │   ├── AdminSlashCommands.cs # Gestion config, jeux, cron
    │   ├── UserSlashCommands.cs # Commandes alternatives
    │   └── InteractionHandlers.cs # Boutons, Menus, Modales
    ├── Program.cs
    └── appsettings.json
```

Modèle de données (Domaine)

```
namespace AssociationBot.Domain.Entities;

public enum ParticipantRole { Player, Spectator }
public enum SessionStatus { Open, Locked, Closed }

public class Game
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public string Name { get; set; } = string.Empty;
    public int? MinPlayers { get; set; }
    public int? MaxPlayers { get; set; }
    public bool IsActive { get; set; } = true;
}

public class GameSession
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public DateTime ScheduledDate { get; set; }
    public ulong DiscordChannelId { get; set; }
    public ulong DiscordMessageId { get; set; }
    public SessionStatus Status { get; set; } = SessionStatus.Open;
```

```
public List<PlayerAvailability> Availabilities { get; set; } = [];  
public List<Table> Tables { get; set; } = [];  
}  
  
public class PlayerAvailability  
{  
    public Guid Id { get; set; } = Guid.NewGuid();  
    public Guid GameSessionId { get; set; }  
    public ulong DiscordUserId { get; set; }  
    public string DiscordUsername { get; set; } = string.Empty;  
    public bool IsAbsent { get; set; }  
    public List<Guid> PreferredGameIds { get; set; } = [];  
    public string? CustomGamePreference { get; set; }  
}  
  
public class Table  
{  
    public Guid Id { get; set; } = Guid.NewGuid();  
    public Guid GameSessionId { get; set; }  
    public Guid? GameId { get; set; }  
    public string CustomGameName { get; set; } = string.Empty;  
    public ulong CreatedByDiscordUserId { get; set; }  
  
    public List<TableParticipant> Participants { get; set; } = [];  
}  
  
public class TableParticipant  
{  
    public Guid Id { get; set; } = Guid.NewGuid();  
    public Guid TableId { get; set; }  
    public ulong DiscordUserId { get; set; }  
    public string DiscordUsername { get; set; } = string.Empty;  
    public ParticipantRole Role { get; set; } = ParticipantRole.Player;  
}
```

Commandes et interactions Discord

Commandes administrateur (/admin-...)

- /admin-config-channel [channel] : Définit le salon de publication automatique.
- /admin-config-schedule [cron_expression] : Définit le créneau de publication (ex. 0 18 * * 1 pour chaque lundi à 18h00).
- /admin-game-add [nom] [min_joueurs] [max_joueurs] : Ajoute un jeu au catalogue.
- /admin-game-list : Liste et permet l'activation/désactivation de jeux.
- /admin-session-create [date] : Force la génération manuelle immédiate d'une session.

Ergonomie utilisateur (Composants Discord sur la Card)

La Card est un message riche (Embed) comportant des boutons et sélecteurs persistants :

1. **Bouton « Déclarer disponibilité »** : Ouvre un composant de sélection multiple (SelectMenu) des jeux actifs, complété d'un bouton « Proposer un autre jeu » qui ouvre une boîte de dialogue (Modal) avec un champ texte.
2. **Bouton « Se déclarer absent »** : Retire le joueur de toute table et met à jour son statut.
3. **Bouton « Créer une table »** : Ouvre une Modal demandant le jeu (ou sélection dans une liste), le type d'inscription (Joueur/Observateur), et l'ajout direct optionnel d'autres membres (via un UserSelectMenu).
4. **Boutons dynamiques sous l'Embed** : Un menu déroulant permettant de rejoindre une table existante en choisissant son rôle (« Rejoindre T1 (Joueur) », « Observer T1 »).
5. **Bouton « Quitter ma table »** : Désassigne l'utilisateur et le replace dans le pool des disponibles.

Structure de l'Embed visuel (La Card)

SESSION DE JEU DU VENDREDI 18 SEPTEMBRE 2026

Statut : Inscriptions ouvertes

DISPONIBILITÉS (Non assignés)

@Alex : SdA, Warhammer 40k

@Bastien : Warhammer 40k, Autre (Dune)

@Chloe : Catan, 7 Wonders

TABLES FORMÉES

Table 1 : Warhammer 40k (1v1)

Joueurs : @Alex, @Bastien

Observateur : @Marc

[Quitter cette table]

Table 2 : Le Seigneur des Anneaux (SdA)

Joueurs : @Thomas, @David

Statut : Complète

ABSENTS : @Julien, @Sophie

Guide de déploiement sur Raspberry Pi

1. Préparation du Raspberry Pi

Un modèle Raspberry Pi 3B+, 4 ou 5 avec un OS 64-bit (Raspberry Pi OS Lite 64-bit ou Ubuntu Server) est requis.

Mettre à jour le système et installer le moteur Docker :

```
sudo apt update && sudo apt upgrade -y  
curl -fsSL https://get.docker.com -o get-docker.sh  
sudo sh get-docker.sh  
sudo usermod -aG docker $USER  
newgrp docker
```

2. Dockerfile multi-stage (.NET 10)

Ce fichier compile l'application pour l'architecture cible du Raspberry Pi (linux-arm64 ou linux-arm).

```
# Étape de compilation  
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build  
WORKDIR /source  
  
COPY *.sln .  
COPY src/AssociationBot.Domain/*.csproj src/AssociationBot.Domain/  
COPY src/AssociationBot.Application/*.csproj src/AssociationBot.Application/  
COPY src/AssociationBot.Infrastructure/*.csproj src/AssociationBot.Infrastructure/  
COPY src/AssociationBot.Presentation/*.csproj src/AssociationBot.Presentation/  
RUN dotnet restore -r linux-arm64  
  
COPY . .  
WORKDIR /source/src/AssociationBot.Presentation  
RUN dotnet publish -c Release -o /app --no-restore -r linux-arm64 --self-contained false  
  
# Image d'exécution finale  
FROM mcr.microsoft.com/dotnet/runtime:10.0 AS final  
WORKDIR /app  
  
# Installation de libsqlite3 pour le runtime natif  
RUN apt-get update && apt-get install -y libsqlite3-0 tzdata && rm -rf /var/lib/apt/lists/*  
ENV TZ=Europe/Paris  
  
COPY --from=build /app .  
VOLUME /app/data  
  
ENTRYPOINT ["dotnet", "AssociationBot.Presentation.dll"]
```

3. Déploiement via Docker Compose

Créer le fichier docker-compose.yml sur le Raspberry Pi :

version: '3.8'

```
services:  
  discord-bot:  
    build:  
      context: .  
      dockerfile: Dockerfile  
    container_name: association_discord_bot  
    restart: unless-stopped  
    environment:  
      - DOTNET_ENVIRONMENT=Production  
      - Discord_Token=VOTRE_BOT_TOKEN_DISCORD  
      - Discord_AdminRoleId=VOTRE_ROLE_ID_ADMIN  
      - ConnectionStrings__DefaultData Source=/app/data/association_bot.db;Cache=Shared  
    volumes:  
      - ./data:/app/data  
    logging:  
      driver: "json-file"  
      options:  
        max-size: "10m"
```

```
max-file: "3"

4. Automatisation des sauvegardes SQLite

Pour éviter toute corruption de données sur la carte SD, planifier une copie à chaud via la CLI SQLite en tâche cron sur l'hôte :

sudo apt install sqlite3 -y
crontab -e

Ajouter la ligne suivante pour exécuter un backup transactionnel sécurisé chaque nuit à 04h00 :

0 4 * * * sqlite3 /chemin/vers/data/association_bot.db ".backup '/chemin/vers/data/backup_${date +%Y/%m/%d}.db'"

5. Mise en route et supervision

Lancer le conteneur en arrière-plan :

docker compose up -d --build

Consulter les journaux applicatifs :

docker compose logs -f --tail=100

Le cycle est opérationnel : l'instance compile de manière optimisée pour ARM64, persiste l'état relationnel dans un volume monté et préserve les composants Discord face aux pannes ou redémarrages de l'hôte.
```

User prompt: 1. Une session s'ouvre et se clôture à la même heure. Si on indique 23h comme heure d'ouverture de session, la session précédente se ferme juste avant l'ouverture de la nouvelle 2. Je ne comprend pas bien la problématique. Mais sur 20 membre d'une asso avec 4-5 commandes à réaliser à la main, je ne suis pas sûr qu'on atteigne la Rate Limits 3. Je ne pense pas qu'on arrive à 25 jeux, mais pourquoi pas se pencher sur l'autocomplétion. 4. Seul le créateur de la table ou un Admin peut dissoudre la table. Un joueur peut uniquement se retirer lui-même. Si le retrait fait passer la table sous deux joueurs, la table est automatiquement détruite immédiatement 5. Avoir l'outil direct, mais notifier le joueur B par message éphémère ou mention avec un bouton d'annulation immédiate en un clic 6. Oui. Pour éviter les conflits d'horaires et d'attention, un utilisateur est soit joueur, soit observateur, mais rattaché à une unique table physique. 7. Utiliser SQLite en mode WAL (Write-Ahead Logging) ou LiteDB avec synchronisation optimisée, avec backup automatisé sur un volume externe ou distant 8. Utiliser des custom_id d'interaction sans état (ex. table:join:sessionId):(tableId) analysés par un routeur global, plutôt que des écouteurs en mémoire éphémère. On s'est mal compris pour la partie LLM. Ce que je demande c'est l'intégration des règles, skills et MCP au sein du projet pour l'assistance ou développement. Par pour un accès externe à une IA qui interroge les données du projet.

Response: Précisions techniques préalables

- **Rate limit Discord sur l'édition de message** : Discord applique un plafond strict de **5 requêtes d'édition par tranche de 5 secondes pour un même message**. Avec 20 membres, le risque est nul en usage étalé, mais réel si plusieurs personnes ouvrent la notification et cliquent à la même seconde. Un simple SemaphoreSlim ou un regroupement d'une seconde suffit sans complexifier le code.
- **Cycle des sessions** : Une tâche planifiée quotidienne/hebdomadaire à heure fixe *H* exécute une transaction atomique : passage de la session S_{n-1} au statut Closed, puis instanciation et envoi de la session S_n .

Configuration de l'environnement de développement assisté par IA

Voici les fichiers à placer à la racine du dépôt pour encadrer strictement le travail des agents IA (Cursor, Windsurf, GitHub Copilot ou assistants compatibles MCP).

1. Fichier de règles contextuelles (.cursorrules ou .github/copilot-instructions.md)

```
# Directives Architecture & Stack

- **Langage / Framework** : C# 10 / .NET 10. Activer Nullable Reference Types ('<Nullable>enable</Nullable>').
- **Bibliothèque Discord** : Discord.Net (version >= 3.17).
- **Paradigme d'architecture** : Clean Architecture stricte.
- 'Domain' : Entités pures, Enums, Interfaces métier. Zéro dépendance vers Discord.Net ou EF Core.
- 'Application' : Cas d'usage (Commands/Queries), DTOs, Interfaces de persistance.
- 'Infrastructure' : EF Core SQLite, configuration Fluent API, Services Discord concrets, Scheduler.
- 'Presentation' : Bot, SlashCommands, Interaction Handlers (boutons, menus, modales).
- **Base de données** : SQLite avec Entity Framework Core. Toujours configurer le mode WAL ('PRAGMA journal_mode = 'wal').

# Règles Métier Invariants

1. **Exclusivité joueur** : Un utilisateur Discord ne peut appartenir qu'à une seule table active par session (en tant que joueur OU observateur).
2. **Seuil critique de table** : Une table requiert au minimum 2 joueurs. Si un désistement fait passer le nombre de joueurs à 1, la table est détruite immédiatement et les participants restants sont reversés dans le statut disponible.
3. **Stateless Interactions** : Aucun état d'interaction en mémoire éphémère. Les 'custom_id' des composants Discord doivent obligatoirement suivre le format délimité : 'contexte:action:param1:param2' (ex: 'table:join:player:(tableId)').
4. **Permissions de dissolution** : Seul le créateur initial de la table ou un utilisateur possédant le rôle Admin configuré peut supprimer manuellement une table.
5. **Assignment directe** : Quand un joueur en assigne un autre sans accord préalable, déclencher une notification mentionnant l'utilisateur assigné avec un bouton d'annulation immédiate ('custom_id="table:leave:(tableId)"').

# Style de Code C#

- Utiliser les Primary Constructors pour l'injection de dépendances.
- Préférer le pattern matching moderne et les expressions switch.
- Méthodes asynchrones : toujours propager le 'CancellationToken'.
- Pas de logique métier dans les handlers de commandes Discord : déléguer systématiquement à la couche Application.
```

2. Configuration des serveurs MCP (.cursor/mcp.json ou configuration agent local)

Cette configuration permet à l'IA d'interroger directement la base SQLite locale de développement, de tester l'état du schéma et de vérifier la cohérence des tables sans avoir à exécuter des scripts manuels.

```
{
  "mcpServers": {
    "sqlite-dev": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-sqlite",
        "--db-path",
        "./src/AssociationBot.Presentation/bin/Debug/net10.0/association_dev.db"
      ]
    },
    "git-operations": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-git"]
    }
  }
}
```

3. Bibliothèque de tâches / Skills pour l'IA (Prompts d'exécution)

À injecter dans l'agent IA selon la brique à implémenter :

- **Skill: Nouvelle Interaction Handler**
« Implémente le traitement de l'interaction pour l'action {ActionName}. Respecte le parsing du custom_id sans état (format: section:action:id). Appelle le Use Case correspondant dans Application. En cas d'erreur de règle métier (ex: joueur déjà assigné), réponds obligatoirement par un message éphémère (ephemeral: true). Mets à jour l'Embed récapitulatif via le service de rendu. »
- **Skill: Migration EF Core**
« Modifie l'entité du domaine {Entity} puis crée la migration EF Core associée. Assure-toi que les configurations Fluent API dans Infrastructure/Persistence/Configurations définissent explicitement les clés étrangères, les suppressions en cascade (Cascade sur TableParticipant, SetNull sur les tables) et les index nécessaires. »

Spécification technique du moteur applicatif

1. Modèle de données affiné (Entity Framework Core)

```
namespace AssociationBot.Domain.Entities;

public enum ParticipantRole { Player, Spectator }
public enum SessionStatus { Open, Closed }

public class GameSession
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public DateTime CreatedAtUtc { get; set; } = DateTime.UtcNow;
    public DateTime ScheduledDate { get; set; }
    public ulong DiscordChannelId { get; set; }
    public ulong DiscordMessageId { get; set; }
    public SessionStatus Status { get; set; } = SessionStatus.Open;

    public List<PlayerAvailability> Availabilities { get; set; } = [];
    public List<GameTable> Tables { get; set; } = [];
}

public class GameTable
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public Guid GameSessionId { get; set; }
    public GameSession GameSession { get; set; } = null!;

    public string GameName { get; set; } = string.Empty;
    public ulong CreatedByDiscordUserId { get; set; }

    public List<TableParticipant> Participants { get; set; } = [];
}

public class TableParticipant
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public Guid GameTableId { get; set; }
    public GameTable GameTable { get; set; } = null!;

    public ulong DiscordUserId { get; set; }
    public string DiscordUsername { get; set; } = string.Empty;
    public ParticipantRole Role { get; set; }
}

public class PlayerAvailability
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public Guid GameSessionId { get; set; }
    public GameSession GameSession { get; set; } = null!;

    public ulong DiscordUserId { get; set; }
```

```

    public string DiscordUsername { get; set; } = string.Empty;
    public bool IsAbsent { get; set; }
    public string PreferredGamesJson { get; set; } = "[]"; // S rialisation des noms/IDs de jeux
}

```

2. Configuration SQLite avec mode WAL (Infrastructure)

```

using Microsoft.Data.Sqlite;
using Microsoft.EntityFrameworkCore;

namespace AssociationBot.Infrastructure.Persistence;

public class AppDbContext(DbContextOptions<AppDbContext> options) : DbContext(options)
{
    public DbSet<GameSession> GameSessions => Set<GameSession>();
    public DbSet<GameTable> GameTables => Set<GameTable>();
    public DbSet<TableParticipant> TableParticipants => Set<TableParticipant>();
    public DbSet<PlayerAvailability> PlayerAvailabilities => Set<PlayerAvailability>();

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        base.OnConfiguring(optionsBuilder);
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.ApplyConfigurationsFromAssembly(typeof(AppDbContext).Assembly);
    }

    public static void InitializeDatabase(string connectionString)
    {
        using var connection = new SqliteConnection(connectionString);
        connection.Open();

        // Activation forc e du mode WAL et synchrone normal pour la performance sur carte SD
        using var command = connection.CreateCommand();
        command.CommandText = @"
            PRAGMA journal_mode = 'wal';
            PRAGMA synchronous = NORMAL;
            PRAGMA foreign_keys = ON;
        ";
        command.ExecuteNonQuery();
    }
}

```

3. Logique m tier critique : Retrait d'un joueur et auto-dissolution (Application)

```

namespace AssociationBot.Application.Tables.Commands;

public class LeaveTableHandler(IAppDbContext db, IDiscordMessageRenderer renderer)
{
    public async Task HandleAsync(Guid tableId, ulong userId, CancellationToken ct)
    {
        var table = await db.GameTables
            .Include(t => t.Participants)
            .Include(t => t.GameSession)
            .FirstOrDefaultAsync(t => t.Id == tableId, ct)
            ?? throw new InvalidOperationException("Table introuvable.");

        var participant = table.Participants.FirstOrDefault(p => p.DiscordUserId == userId)
            ?? throw new InvalidOperationException("Le joueur ne fait pas partie de cette table.");

        table.Participants.Remove(participant);

        // R gle d'auto-destruction : Moins de 2 joueurs => dissolution
        int remainingPlayers = table.Participants.Count(p => p.Role == ParticipantRole.Player);

        if (remainingPlayers < 2)
        {
            db.GameTables.Remove(table);
        }

        await db.SaveChangesAsync(ct);
        await renderer.QueueMessageUpdateAsync(table.GameSessionId, ct);
    }
}

```

4. Routeur d'interactions sans  tat (Presentation)

```

using Discord.WebSocket;

namespace AssociationBot.Presentation.Discord;

public class InteractionRouter(
    IServiceProvider services,
    DiscordSocketClient client)
{
    public void Initialize()
    {
        client.ButtonExecuted += HandleButtonAsync;
    }

    private async Task HandleButtonAsync(SocketMessageComponent component)
    {
        // Format attendu: module:action:param1:param2
        var tokens = component.Data.CustomId.Split(':');
        if (tokens.Length < 2) return;

        var (module, action) = (tokens[0], tokens[1]);

        switch (module)
        {
            case "table":
                await HandleTableInteractionAsync(component, action, tokens.Skip(2).ToArray());
                break;
            case "session":
                await HandleSessionInteractionAsync(component, action, tokens.Skip(2).ToArray());
                break;
        }
    }

    private async Task HandleTableInteractionAsync(SocketMessageComponent component, string action, string[] args)
    {
        switch (action)
        {
            case "leave":
                var tableId = Guid.Parse(args[0]);
                // R solution sc p e du handler applicatif
                using (var scope = services.CreateScope())
                {
                    var handler = scope.ServiceProvider.GetRequiredService<LeaveTableHandler>();
                    await handler.HandleAsync(tableId, component.User.Id, CancellationToken.None);
                }
                await component.RespondAsync("Tu as quitt  la table.", ephemeral: true);
                break;

            case "join":
                // args[0] = role ("player" ou "spectator"), args[1] = tableId
                break;
        }
    }

    private Task HandleSessionInteractionAsync(SocketMessageComponent component, string action, string[] args) => Task.CompletedTask;
}

```

Guide de mise en service sur Raspberry Pi

1. Configuration des volumes et variables

Structure des dossiers sur le Raspberry Pi (/opt/discord-bot):

```

/opt/discord-bot/
├─ docker-compose.yml
├─ backup.sh
└─ data/
    └─ (association_bot.db g n r  ici)

```

2. Fichier docker-compose.yml

```

version: '3.8'

services:
  bot:
    image: association-bot:latest
    build:
      context: .

```

```
dockerfile: Dockerfile
container_name: asso_discord_bot
restart: unless-stopped
environment:
  - DOTNET_ENVIRONMENT=Production
  - Discord_Token=VOTRE_TOKEN_BOT
  - Discord_AdminRoleId=123456789012345678
  - ConnectionStrings__Default=Data Source=/app/data/association_bot.db
volumes:
  - ./data:/app/data
logging:
  driver: "json-file"
  options:
    max-size: "10m"
    max-file: "3"
```

3. Script de sauvegarde transactionnelle (backup.sh)

Placer ce script dans /opt/discord-bot/backup.sh :

```
#!/bin/bash
BACKUP_DIR="/opt/discord-bot/backups"
DB_FILE="/opt/discord-bot/data/association_bot.db"
DATE=$(date +"%Y%m%d_%H%M%S")

mkdir -p "$BACKUP_DIR"

# Sauvegarde à chaud non bloquante compatible mode WAL
sqlite3 "$DB_FILE" ".backup '$BACKUP_DIR/bot_${DATE}.db'"

# Rétention : suppression des sauvegardes de plus de 14 jours
find "$BACKUP_DIR" -name "bot_*.db" -mtime +14 -delete

Rendre le script exécutable et planifier l'exécution quotidienne via crontab :

chmod +x /opt/discord-bot/backup.sh
(crontab -l 2>/dev/null; echo "0 3 * * * /opt/discord-bot/backup.sh") | crontab -
```

Le code et l'infrastructure sont prêts pour une implémentation directe, assistée par IA via le serveur MCP SQLite et les règles d'architecture formalisées.

User prompt: *il me faudrait un nom pour ce projet ctr-xxx (CTR pour l'acronyme de l'asso)*

Response: **Fonctionnel & Organisation** (clair, orienté dev et objectif)

- **ctr-roster** : référence directe aux listes de joueurs et d'armées, parfaitement adapté au croisement wargame/jeu de société.
- **ctr-tables** : minimaliste et sans équivoque sur la finalité du bot.
- **ctr-session** : met l'accent sur l'événement et son cycle de vie temporel.

Thématique JDR & Ambiance (rôle d'organisateur au sein du club)

- **ctr-intendant** : évoque la logistique, la gestion des ressources et l'attribution des places.
- **ctr-scribe** : celui qui consigne les présences, les disponibilités et les configurations de jeu.
- **ctr-taverne** : le lieu de ralliement classique où l'on forme des groupes avant de partir en partie.

Thématique Wargame & Stratégie (ton martial et tactique)

- **ctr-deploy** : axé sur le placement des effectifs sur les différentes tables de jeu.
- **ctr-hq** (ou **ctr-gg**) : le quartier général où se centralisent les forces en présence.
- **ctr-warroom** : la salle où se préparent les affrontements de la soirée.

Pour un nom technique propre sur GitHub et un pseudo clair sur Discord, **ctr-roster** ou **ctr-intendant** offrent le meilleur compromis entre clarté fonctionnelle et identité ludique.